

数据库实验报告（第六周）

班 级：3班

姓 名：梁力航

学 号：23336128

一、实验目的

- 熟悉SQL的有关视图的操作
- 能够使用SQL语句创建需要的视图，对视图进行查询和取消视图

二、实验内容

1. 定义选课信息和课程名称的视图 VIEWC
2. 定义学生姓名与选课信息的视图 VIEWS
3. 定义年级低于1998的学生的视图S1(SID, SNAME, GRADE)
4. 查询学生为"uxjof"的学生的选课信息
5. 查询选修课程"UML"的学生的编号和成绩
6. 向视图S1插入记录(60000001,Lily,2001)
7. 定义包括更新和插入约束的视图S1，尝试向视图插入记录(60000001,Lily,1997)，删除所有年级为1999的学生记录，讨论更新和插入约束带来的影响
8. 在视图VIEWS中将姓名为"uxjof"的学生的选课成绩都加上5分
9. 取消以上建立的所有视图

说明：所有操作均在 USE School_Data; 上下文下执行。

三、实验结果

(1) 定义选课信息和课程名称的视图 VIEWC

```
DROP VIEW IF EXISTS VIEWC;
GO

CREATE VIEW VIEWC AS
SELECT CHOICES.*, COURSES.cname
FROM CHOICES
JOIN COURSES ON CHOICES.cid = COURSES.cid;
GO

SELECT * FROM VIEWC;
```

```

10  -- (1)定义选课信息和名称的视图 VIEWC;
11  ✓ drop view if exists VIEWC;
12  go
13
14  ✓ create view VIEWC as
15  select CHOICES.*, COURSES.cname
16  from CHOICES
17       join COURSES 1..n<->1: on CHOICES.cid = COURSES.cid;
18  go
19
20  ✓ select * from VIEWC;|
21
22  -- (2)定义学生姓名与选课信息的视图 VIEWS

```

输出 Result 28-3

	no	sid	tid	cid	score	cname
1	500000058	823069829	249596497	10037		76 software testing
2	500000253	829348273	202560416	10010		87 software engineering
3	500000984	850955252	234145610	10021		54 j2me
4	500001270	847061074	292043491	10025		92 embedded system
5	500002143	860635914	238811498	10039		82 fortran
6	500002371	829785562	273189968	10028		77 architectonics
7	500002715	822137137	218922066	10011		67 distributed computing
8	500003262	826310502	267846042	10005		90 c++

299,550 行

(2) 定义学生姓名与选课信息的视图 VIEWS

```

DROP VIEW IF EXISTS VIEWS;
GO

CREATE VIEW VIEWS AS
SELECT STUDENTS.sname, CHOICES.*
FROM CHOICES
     JOIN STUDENTS ON CHOICES.sid = STUDENTS.sid;
GO

SELECT * FROM VIEWS;

```

```
-- (2)定义学生姓名与选课信息的视图 VIEWS;

drop view if exists VIEWS;
go

create view VIEWS as
select STUDENTS.sname, CHOICES.*
from CHOICES
    join STUDENTS 1..n<->1: on CHOICES.sid = STUDENTS.sid;
go

select * from VIEWS;
```

输出 Result 29-3

	sname	no	sid	tid	cid	score
1	fmnlxsl	502175939	890035888	269801126	10050	84
2	nnsdolcvw	502176924	866594050	211035569	10025	94
3	ykytcjhij	502179848	831920355	212114441	10006	83
4	wsmqx	502181871	818879765	299206520	10033	74
5	qokgc	502183605	800417342	202560861	10002	63
6	blhtdve	502186765	875847452	209288173	10025	62
7	uubyxbu	502187674	811742937	2774523		55
8	jabsnx	502194421	830538722	213070137	10010	68

(3) 定义年级低于1998的学生的视图S1(SID, SNAME, GRADE)

```
DROP VIEW IF EXISTS S1;
GO

CREATE VIEW S1 (SID, SNAME, GRADE) AS
SELECT sid, sname, grade
FROM STUDENTS
WHERE grade > 1998;
GO

SELECT * FROM S1;
```

括号内的内容是用于命名视图中的列

```

34  --(3)定义年级低于1998的学生的视图S1(SID, SNAME, GRADE);
35  ✓ drop view if exists S1;
36  go
37
38  ✓ create view S1 (SID, SNAME, GRADE) as
39  select SID sid, SNAME sname, GRADE grade
40  from STUDENTS
41  where
42      grade > 1998;
43  go
44
45  ✓ select * from S1;

```

输出 Result 46-3 ×



	SID	SNAME	GRADE
1	800002933	vnbgzsvv	2002
2	800008565	ehlycg	1999
3	800009026	rcxaihj	2002
4	800013889	nahhluoe	2000
5	800015244	egwnnf	1999
6	800023487	tkbzqduq	1999
7	800028044	ztozlv	2001

40,092 行

(4) 查询学生为"uxjof"的学生的选课信息

```
SELECT * FROM VIEWS WHERE sname = 'uxjof';
```

```

47  --(4)查询学生为"uxjof"的学生的选课信息;
48  ✓ select * from VIEWS where sname = 'uxjof';
49

```

输出 (4)查询学生为"uxjof"的学生的选课信息; ×



	sname	no	sid	tid	cid	score
1	uxjof	541221076	800023963	238341990	10018	84
2	uxjof	506978093	800023963	220667042	10046	94
3	uxjof	567316431	800023963	258375444	10037	98

3 行


```

3      -- (6)向视图S1插入记录(600000001,Lily,2001);
4      -- 问题：视图S1定义为grade<1998，但插入的grade=2001不满足条件
5      -- 如果视图没有WITH CHECK OPTION，插入会成功但在视图中看不到
6      -- 如果视图有WITH CHECK OPTION，插入会失败
7
8      -- 先清理可能存在的测试数据
9      delete from STUDENTS where sid = '600000001';
10
11     -- 查看插入前的视图内容
12     select * from S1 where SID = '600000001';

```

输出
查看插入前的视图内容 ×
查看插入前的基表内容
查看插入后的视图内容（看不到gr

SID

SNAME

GRADE

0 行

-- 查看插入前的基表内容

```
SELECT * FROM STUDENTS WHERE sid = '600000001';
```

54

-- 查看插入前的基表内容

55

select * from STUDENTS where sid = '60000001';

56

输出

查看插入前的视图内容

查看插入前的基表内容

查看插入后的视图内容（看不到grade=2001的记录）

sid

sname

email

grade

0行

-- 执行插入操作

```
INSERT INTO S1 (SID, SNAME, GRADE) VALUES ('60000001', 'Lily', 2001);
```

-- 查看插入后的视图内容（可以看到grade=2001的记录，因为满足视图条件grade>1998）

```
SELECT * FROM S1 WHERE SID = '60000001';
```

70

--6查看插入后的视图内容

71

select * from S1 where SID = '60000001';

72

L"的学生的编号和成绩;

6查看插入前的视图内容

6查看插入前的基表内容

6查看插入后的视图内容

SID

SNAME

GRADE

1	60000001	Lily	2001
---	----------	------	------

1行

-- 查看插入后的基表内容（可以看到记录已插入）

```
SELECT * FROM STUDENTS WHERE sid = '60000001';
```

67

-- 执行插入操作

68

insert into S1 (SID, SNAME, GRADE) values (SID '60000001', SNAME 'Lily', GRADE 2001);

69

70

-- 查看插入后的视图内容（看不到grade=2001的记录，因为不满足视图条件）

71

select * from S1 where SID = '60000001';

72

73

-- 查看插入后的基表内容（可以看到记录已插入）

74

select * from STUDENTS where sid = '60000001';

75

查看插入前的视图内容

查看插入前的基表内容

查看插入后的视图内容（看不到grade=2001的记录，因为不满足视图条件）

查看插入后的基表内容（可以看到记录已插入）

sid

sname

email

grade

1

60000001

Lily

<null>

2001

1行

说明：视图S1定义为grade>1998，插入的grade=2001满足条件。由于视图没有WITH CHECK OPTION，插入成功并且在视图和基表中都可以看到该记录。

(7) 定义包括更新和插入约束的视图S1

```
-- 先清理测试数据
DELETE FROM STUDENTS WHERE sid = '60000001';

-- 删除原来的S1视图
DROP VIEW IF EXISTS S1;
GO

-- 重新创建带WITH CHECK OPTION的视图
CREATE VIEW S1 (SID, SNAME, GRADE) AS
SELECT sid, sname, grade
FROM STUDENTS
WHERE grade > 1998
WITH CHECK OPTION;
GO
```

操作前状态

```
-- 查看操作前-视图S1中sid=60000001的记录
SELECT * FROM S1 WHERE SID = '60000001';
```



```
94 --7操作前-视图S1中sid=60000001的记录
95 ✓ select * from S1 where SID = '60000001';
96
```

输出 7操作前-视图S1中sid=60000001的记录 ×



☐ SID ☐ SNAME ☐ GRADE

0行 ▾ | ⋮

-- 查看操作前-基表中grade=1999的记录数

```
SELECT COUNT(*) AS 记录数 FROM STUDENTS WHERE grade = 1999;
```

```
97 --7操作前-基表中grade=1999的记录数
98 ✓ select count(*) as 记录数 from STUDENTS where grade = 1999;
99
```

输出 7操作前-基表中grade=1999的记录数 ×



☐ 记录数 ▾

1	6664
---	------

1行 ▾ | ⋮

插入操作

-- 尝试插入记录(60000001,Lily,1997) - 应该失败, 因为1997不满足grade>1998

-- 由于有WITH CHECK OPTION, 这个插入操作会被拒绝

```
INSERT INTO S1 (SID, SNAME, GRADE) VALUES ('60000001', 'Lily', 1997);
```

```
99
100 -- 尝试插入记录(60000001,Lily,1997) - 应该成功, 因为1997<1998
101 ! insert into S1 (SID, SNAME, GRADE) values ( SID '60000001', SNAME 'Lily', GRADE 1997);
102
103 --7插入后-视图S1中sid=60000001的记录
104 select * from S1 where SID = '60000001';
105
```

[S0001][550] 行 1: The attempted insert or update failed because the target view either specifies WITH CHECK OPTION or spans a view that specifies WITH CHECK OPTION and one or more rows resulting from the operation did not qualify under the CHECK OPTION constraint.

说明：由于WITH CHECK OPTION的约束，插入grade=1997的记录被拒绝，因为不满足视图定义的条件grade>1998。

-- 查看插入后-视图S1中sid=60000001的记录
SELECT * FROM S1 WHERE SID = '60000001';

103

104 --7插入后-视图S1中sid=60000001的记录

105 ✓ select * from S1 where SID = '60000001';

106

输出 7插入后-视图S1中sid=60000001的记录 7操作前-基表中grade=1999的记录数

SQL Editor

SID SNAME GRADE

0行

-- 查看插入后-基表中sid=60000001的记录
SELECT * FROM STUDENTS WHERE sid = '60000001';

L00

L07

L08 ✓

L09

L10

-- 7插入后-基表中sid=60000001的记录
select * from STUDENTS where sid = '60000001';
-- 删除所有年级为1999的学生记录

输出

7插入后-视图S1中sid=60000001的记录

7插入后-基表中sid=60000001的记录 ×

+

—

↶

↷

↕

↑

Tx: 自动 ▾

DDL

📌

🔍

📄

🔗

🔗 sid ▾

🔗 sname ▾

🔗 email ▾

🔗 grade ▾

0 行 ▾ | ⋮

删除操作

由于STUDENTS表与CHOICES表之间存在外键约束，直接通过视图删除学生记录会失败。需要先删除CHOICES表中的相关选课记录，然后才能删除学生记录。

-- 先删除CHOICES表中grade=1999学生的选课记录

```
DELETE FROM CHOICES
WHERE sid IN (
    SELECT sid
    FROM STUDENTS
    WHERE STUDENTS.grade = 1999
);
```

-- 然后删除STUDENTS表中grade=1999的学生记录

```
DELETE FROM S1 WHERE GRADE = 1999;
```

-- 查看删除后-视图中grade=1999的记录数

```
SELECT COUNT(*) AS 记录数 FROM S1 WHERE grade = 1999;
```

```
123 --7删除后-视图中grade=1999的记录数
124 ✓ select count(*) as 记录数 from STUDENTS where grade = 1999;
125 |
126 --7删除后-基表中grade=1999的记录数
127 select count(*) as 记录数 from STUDENTS where grade = 1999;
128
```

输出 7删除后-视图中grade=1999的记录数 7插入后-基表中sid=60000001的记录

记录数

1	0
---	---

1行

-- 查看删除后-基表中grade=1999的记录数
SELECT COUNT(*) AS 记录数 FROM STUDENTS WHERE grade = 1999;

```

126 -- 7删除后-基表中grade=1999的记录数
127 ✓ select count(*) as 记录数 from STUDENTS where grade = 1999;
128
129 -- 讨论: WITH CHECK OPTION确保通过视图插入或更新的数据必须满足视图的WHERE条件
130 -- 1. 插入grade=1997失败, 因为不满足grade>1998, WITH CHECK OPTION会拒绝此操作

```

输出

7删除后-视图中grade=1999的记录数

7删除后-基表中grade=1999的记录数

记录数

1	0
---	---

1行

讨论:

- WITH CHECK OPTION确保通过视图插入或更新的数据必须满足视图的WHERE条件
- 插入grade=1997失败, 因为不满足grade>1998, WITH CHECK OPTION会拒绝此操作
- 删除grade=1999的记录时, 由于外键约束, 需要先删除CHOICES表中的相关记录, 然后才能通过视图删除STUDENTS表中的记录
- 删除操作成功后, 视图和基表中grade=1999的记录数都变为0

(8) 在视图VIEWS中将姓名为"uxjof"的学生的选课成绩都加上5分

-- 先备份原始成绩到临时表

```

IF OBJECT_ID('tempdb..#uxjof_backup') IS NOT NULL
    DROP TABLE #uxjof_backup;

SELECT C.no, C.sid, C.tid, C.cid, C.score AS original_score
INTO #uxjof_backup
FROM CHOICES C
JOIN STUDENTS S ON C.sid = S.sid
WHERE S.sname = 'uxjof';

```

更新前

```
SELECT * FROM VIEWS WHERE sname = 'uxjof';
```

132

--8更新前-uxjof的选课成绩

133

```
select * from VIEWS where sname = 'uxjof';
```

134

输出

8更新前-uxjof的选课成绩

8更新后-uxjof的选课成绩

8更新后-基表CHOICES中的对应记录

8恢复后-uxjof的选课成绩已恢复到原始值

sname

no

sid

tid

cid

score

1

uxjof

506978093

800023963

220667042

10046

94

2

uxjof

541221076

800023963

238341990

10018

84

3

uxjof

567316431

800023963

258375444

10037

98

3行

执行更新

```
UPDATE VIEWS SET score = score + 5 WHERE sname = 'uxjof';

SELECT * FROM VIEWS WHERE sname = 'uxjof';
```

135

-- 执行更新操作

136

```
update VIEWS set score = score + 5 where sname = 'uxjof';
```

137

138

--8更新后-uxjof的选课成绩

139

```
select * from VIEWS where sname = 'uxjof';
```

输出

8更新前-uxjof的选课成绩

8更新后-uxjof的选课成绩

8更新后-基表CHOICES中的对应记录

8恢复后-uxjof的选课成绩已恢复到原始值

sname

no

sid

tid

cid

score

1

uxjof

567316431

800023963

258375444

10037

103

2

uxjof

506978093

800023963

220667042

10046

99

3

uxjof

541221076

800023963

238341990

10018

89

3行

验证基表

```
SELECT C.*
FROM CHOICES C
JOIN STUDENTS S ON C.sid = S.sid
WHERE S.sname = 'uxjof';
```

141 --8更新后-基表CHOICES中的对应记录

142 select C.*

143 from CHOICES C

144 join STUDENTS S 1..n<->1: on C.sid = S.sid

145 where S.sname = 'uxjof';

输出8更新前-uxjof的选课成绩8更新后-uxjof的选课成绩8更新后-基表CHOICES中的对应记录8恢复后-uxjof的选课成绩已恢复到原始值

no

sid

tid

cid

score

15069780938000239632206670421004699

25412210768000239632383419901001889

356731643180002396325837544410037103

3行

恢复原始数据

```
-- 恢复原始成绩（避免反复运行导致成绩越来越高）
UPDATE C
SET C.score = B.original_score
FROM CHOICES C
JOIN #uxjof_backup B ON C.no = B.no;

SELECT * FROM VIEWS WHERE sname = 'uxjof';
```

147 -- 恢复原始成绩（避免反复运行导致成绩越来越高）

148 update C

149 set C.score = B.original_score

150 from CHOICES C

151 join #uxjof_backup B on C.no = B.no;

152

153 --8恢复后-uxjof的选课成绩已恢复到原始值

154 select * from VIEWS where sname = 'uxjof';

输出8更新前-uxjof的选课成绩8更新后-uxjof的选课成绩8更新后-基表CHOICES中的对应记录8恢复后-uxjof的选课成绩已恢复到原始值

sname

no

sid

tid

cid

score

1uxjof5069780938000239632206670421004694

2uxjof5673164318000239632583754441003798

3uxjof5412210768000239632383419901001884

说明：通过视图更新数据会同时更新基表。为避免反复运行脚本导致成绩累加，使用临时表备份并在演示后恢复原始数据。

(9) 取消以上建立的所有视图

```
-- 查看删除前-当前数据库中的视图
SELECT TABLE_NAME AS 视图名称
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_NAME IN ('VIEWC', 'VIEWS', 'S1');
```

```
156 -- (9)取消以上建立的所有视图。
157 ✓ -- 9删除前-当前数据库中的视图
158 select TABLE_NAME as 视图名称
159 from INFORMATION_SCHEMA.VIEWS
160 where
161     TABLE_NAME in ('VIEWC', 'VIEWS', 'S1');
162
```

输出 9删除前-当前数据库中的视图 9删除后-当前数据库中的视图

视图名称

1	S1
2	VIEWC
3	VIEWS

3行

```
-- 删除视图
DROP VIEW IF EXISTS VIEWC;
DROP VIEW IF EXISTS VIEWS;
DROP VIEW IF EXISTS S1;

-- 查看删除后-当前数据库中的视图
SELECT TABLE_NAME AS 视图名称
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_NAME IN ('VIEWC', 'VIEWS', 'S1');
```



```
163 -- 删除视图
164 drop view if exists VIEWC;
165
166 drop view if exists VIEWS;
167
168 drop view if exists S1;
169
170 --9删除后-当前数据库中的视图
171 select TABLE_NAME as 视图名称
172 from INFORMATION_SCHEMA.VIEWS
173 where
174     TABLE_NAME in ('VIEWC', 'VIEWS', 'S1');
```

输出 9删除前-当前数据库中的视图 9删除后-当前数据库中的视图 ×

视图名称

0行

四、问题&总结

通过本次实验，我对视图的概念和使用有了更深入的理解。视图本质上是一个虚拟表，它不存储实际数据，只保存查询定义。当我们查询视图时，数据库会将视图的查询转换为对基表的查询。这种机制既可以简化复杂的查询操作，又能提供一定的数据安全性和逻辑独立性。

在第6题和第7题的对比中，我发现WITH CHECK OPTION这个约束的作用非常明显。视图S1定义为年级低于1998（即 $grade > 1998$ ，因为年份越大年级越低）。在第6题中，视图没有WITH CHECK OPTION约束，插入 $grade = 2001$ 的记录成功了，因为 $2001 > 1998$ 满足视图条件，所以在视图和基表中都能看到。但在第7题中，加上WITH CHECK OPTION后，尝试插入 $grade = 1997$ 的记录就会失败，因为1997不满足 $grade > 1998$ 的条件，约束会直接拒绝这个操作。这个特性在实际应用中很有用，可以确保通过视图操作的数据都符合预期的条件。

在删除操作中遇到了一个实际问题。虽然 $grade = 1999$ 满足视图条件（ $1999 > 1998$ ），理论上可以通过视图删除，但由于STUDENTS表与CHOICES表之间存在外键约束，直接删除学生记录会违反参照完整性。这让我意识到，通过视图进行删除操作时，不仅要考虑视图本身的条件，还要考虑基表之间的约束关系。解决方法是先删除CHOICES表中该学生的选课记录，然后再通过视图删除STUDENTS表中的学生记录。这个过程体现了数据库完整性约束的重要性。

关于视图的更新，我发现并不是所有视图都可以随意更新。像S1这种简单的行列子集视图是可以更新的，但基于多表连接的视图（如VIEWC、VIEWS）更新起来就有限制了，通常只能更新其中一个基表的数据。如果视图包含聚合函数、GROUP BY、DISTINCT或UNION这些复杂操作，那就完全不能更新了。第8题中我通过VIEWS视图更新了学生成绩，发现基表CHOICES也同步更新了，这说明对视图的修改会直接作用到基表上。

在编写SQL语句时遇到了一些语法问题。SQL Server要求CREATE VIEW必须是批处理中的第一条语句，所以需要GO语句来分隔不同的批处理。一开始我没注意这点，导致报错。后来在每个DROP VIEW和CREATE VIEW之间加上GO就解决了。另外使用DROP VIEW IF EXISTS可以避免重复创建视图时的错误，这样脚本可以反复运行而不会出问题。

第9题删除视图的操作让我意识到，删除视图只是删除了查询定义，并不会影响基表的数据。但如果有其他视图是基于被删除的视图创建的，那些视图就会失效，需要手动删除。通过查询INFORMATION_SCHEMA.VIEWS系统视图，可以方便地查看数据库中存在哪些视图。

在实践过程中，我还总结了一些小技巧。比如为了避免测试数据污染数据库，在插入测试记录前先清理可能存在的旧记录。对于会修改数据的操作，可以用临时表先备份原始数据，演示完成后再恢复，这样就不会因为反复运行脚本而导致数据异常（比如第8题中成绩不断累加的问题）。